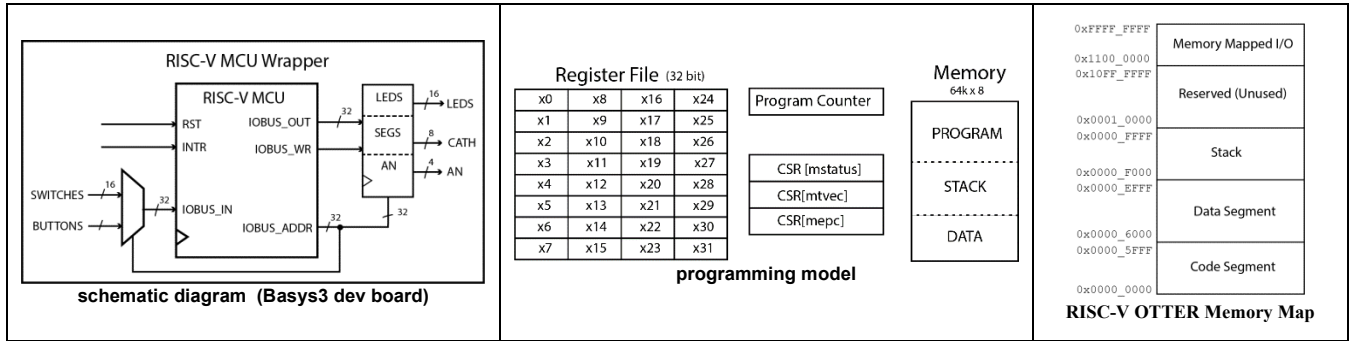


RISC-V MCU Architecture and Assembly Language Cheat Sheet

V2.15 © Copyright: 2025 james mealy



RISC-V OTTER Instruction Set: (shaded indicate pseudoinstructions)

Program Control		
jal rd,imm	j imm	jal imm
jalr rd,rs1,imm	jr rs	jalr rs
call imm	ret	mret
beq rs1,rs2,imm	beqz rs1,imm	
bne rs1,rs2,imm	bnez rs1,imm	
blt rs1,rs2,imm	blez rs1,imm	bgt rs1,rs2,imm
bge rs1,rs2,imm	bgez rs1,imm	ble rs1,rs2,imm
bltu rs1,rs2,imm	bltz rs1,imm	bgtu rs1,rs2,imm
bgeu rs1,rs2,imm	bgtz rs1,imm	bleu rs1,rs2,imm

Load/Store (& I/O)		
lb rd,imm(rs1)		sb rs2,imm(rs1)
lh rd,imm(rs1)		sh rs2,imm(rs1)
lw rd,imm(rs1)	lw rd,imm	sw rs2,imm(rs1)
lbu rd,imm(rs1)		
lhu rd,imm(rs1)		
Operations		
addi rd,rs1,imm	add rd,rs1,rs2	
	sub rd,rs1,rs2	neg rd,rs1
xori rd,rs1,imm	xor rd,rs1,rs2	not rd,rs1
ori rd,rs1,imm	or rd,rs1,rs2	
andi rd,rs1,imm		
slli rd,rs1,imm	sll rd,rs1,rs2	
srl rd,rs1,imm	srl rd,rs1,rs2	sgtz rd,rs1
srai rd,rs1,imm	sra rd,rs1,rs2	sltz rd,rs1
slti rd,rs1,imm	slt rd,rs1,rs2	snez rd,rs1
sltiu rd,rs1,imm	sltu rd,rs1,rs2	seqz rd,rs1
Auxiliary		
nop	auipc rd,imm	lui rd,imm
csrrw rd,csr,rs1	li rd,imm	mv rd,rs
la rd,imm	csrrc rd,csr,rs1	csrrs rd,csr,rs1

Register Designations:

reg	ABI	reg	ABI	reg	ABI	reg	ABI
x0	0	x8	so/fp	x16	a6	x24	s8
x1	ra	x9	s1	x16	a7	x25	s9
x2	sp	x10	a0	x18	s2	x26	s10
x3	gp	x11	a1	x19	s3	x27	s11
x4	fp	x12	a2	x20	s4	x28	t3
x5	t0	x13	a3	x21	s5	x29	t4
x6	t1	x14	a4	x22	s6	x30	t5
x7	t2	x15	a5	x23	s7	x31	t6

x0 read only: set to zero

Interrupt Architecture:

Firmware:

Init: csrrw x4,mtvec,x6 # store ISR addr
csrrs x4,mstatus,x1 # enable intrrs

Exit ISR mret

Hardware:

On interrupt: CSR[mstatus[MPIE]] ← CSR[mstatus[MIE]]
CSR[mstatus[MIE]] ← 0 (mask interrupt)
CSR[mepc] ← PC (save PC)
PC ← CSR[mtvec] (load interrupt vector)

Exiting ISR: PC ← CSR[mepc] (load return address)
CSR[mstatus[MIE]] ← CSR[mstatus[MPIE]]

dir_1: .word 0x23, 0x47 # also .half, .byte
dir_2: .space 34 # room for this many bytes
.text # instructions follow this
.data # data follows this (.word, .half, .byte)

- Max program size: ~16k instructions (32-bit/instr)
- 32 32-bit general purpose registers (GPRs)
- Register x0: read only; always zero
- Register x1: return address
- Register x2: stack pointer
- Stack: implemented in memory

I/O (memory mapped): addr ≥ 0x0001_0000

Input

```
li x10,0xC0000004 # load port addr in reg
lw x20,0(x10) # input data to x20
```

Output

```
li x10,0xC0000008 # load port addr in reg
sw x20,0(x10) # output data in x20
```

bit setting: OR with '1'	bit clearing: AND with '0'	bit toggling: XOR with '1'
ori x8,x8,0xF ;set 4 LSBs	andi x8,x8,0xF clear 28 MSBs	xori x8,x8,0x0F ;toggle 4 LSBs

Useful Constructs:

PUSH (store word on stack)		POP (copy word from stack)	
addi sp,sp,-4 # decrease stack pointer		lw x8,0(sp) # copy value to x8	
sw x8,0(sp) # copy x8 to stack		addi sp,sp,4 # increase stack pointer	
if/else construct			
"If" x20=0, jump to zero; "Else", do something, then jump over "if"	else:	beq x20,x0,if # check cond: branch when met (if)	
		add x27,x28,x29 # -- "else" (dummy instruction)	
		j done # jump over "if"	
	if:	sub x17,x18,x19 # -- "if" (dummy instruction)	
done:		nop # keep doing meaningful stuff	
iterative construct (unknown iterative #: do-while)			
initialize x22 with check value add to count each iteration check result	init:	li x22,923 # load some value to x20	
		mv x20,x0 # clear register	
	loop:	add x20,x20,x23 # add some value	
		blt x20,x22,loop # repeat if sum is < x22	
loop_done:		nop # do something else...	
iterative construct (known iterative value: while)			
Initial loop count	init:	li x20,10 # initialize loop count	
	loop:	beq x20,x0,loop_done # check stopping condition	
		add x4,x5,x6 # -- dummy instr (do something)	
		addi x20,x20,-1 # decrement iteration variable	
		j loop # branch to continue	
	loop_done:	nop # do something else...	

Instruction	Description	RTL	Comment
add rd,rs1,rs2	addition	$X[rd] \leftarrow X[rs1] + X[rs2]$	
addi rd,rs1,imm	addition with immediate	$X[rd] \leftarrow X[rs1] + \text{sext}(imm)$	
and rd,rs1,rs2	bitwise AND	$X[rd] \leftarrow X[rs1] \cdot X[rs2]$	
andi rd,rs1,imm	Bitwise AND immediate	$X[rd] \leftarrow X[rs1] \cdot \text{sext}(imm)$	
auipc rd,imm	add upper immediate to PC	$X[rd] \leftarrow PC + (\text{sext}(imm) < 12)$	
beq rs1,rs2,imm	branch if equal	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] == X[rs2])$	imm ≠ value
beqz rs1,imm	branch if equal to zero	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] == 0)$	imm ≠ value
bge rs1,rs2,imm	branch if greater than or equal	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \geq_s X[rs2])$	imm ≠ value
bgeu rs1,rs2,imm	branch if greater than or equal unsigned	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \geq_u X[rs2])$	imm ≠ value
bgez rs1,imm	branch if greater than or equal to zero	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \geq_s 0)$	imm ≠ value
bgt rs1,rs2,imm	branch if greater than	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] >_s X[rs2])$	imm ≠ value
bgtu rs1,rs2,imm	branch if greater than unsigned	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] >_u X[rs2])$	imm ≠ value
bgtz rs1,rs2,imm	branch if greater than zero	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] >_s 0)$	imm ≠ value
ble rs1,rs2,imm	branch if less than or equal	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \leq_s X[rs2])$	imm ≠ value
bleu rs1,rs2,imm	branch if less than or equal (unsigned)	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \leq_u X[rs2])$	imm ≠ value
blez rs1,rs2,imm	branch if less than or equal zero	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \leq_s 0)$	imm ≠ value
blt rs1,rs2,imm	branch if less than	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] <_s X[rs2])$	imm ≠ value
bltz rs1,imm	branch if less than zero	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] <_s 0)$	imm ≠ value
bltu rs1,rs2,imm	branch if less than (unsigned)	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] <_u X[rs2])$	imm ≠ value
bne rs1,rs2,imm	branch if not equal	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \neq X[rs2])$	imm ≠ value
bnez rs1,imm	branch if not equal to zero	$PC \leftarrow PC + \text{sext}(imm)$ if $(X[rs1] \neq 0)$	imm ≠ value
call label	branch to subroutine WARNING: overwrites X6	$X[rd] \leftarrow PC + 8$; $PC \leftarrow \&\text{symbol}$ (rd=X1 if rd omitted)	imm ≠ value overwrites X6
csrrc rd,csr,rs1	control & status reg read and bit clear	$X[rd] \leftarrow \text{CSR}[csr]$; $\text{CSR}[csr] \leftarrow \text{CSR}[csr] \& \sim rs1$	clears part of reg
csrrs rd,csr,rs1	control & status reg read and bit set	$X[rd] \leftarrow \text{CSR}[csr]$; $\text{CSR}[csr] \leftarrow \text{CSR}[csr] rs1$	sets part of reg
csrrw rd,csr,rs1	control & status register read & write	$X[rd] \leftarrow \text{CSR}[csr]$; $\text{CSR}[csr] \leftarrow rs1$	writes entire reg
csrw rs1,csr	control & status register write	$\text{CSR}[csr] \leftarrow rs1$	
j imm	unconditional branch	$PC \leftarrow PC + \text{sext}(imm)$	imm ≠ value
jal rd,imm	unconditional branch with offset	$X[rd] \leftarrow PC + 4$; $PC \leftarrow PC + \text{sext}(imm)$	imm ≠ value rd=X1 if rd omitd
jalr rd,rs1,imm	unconditional branch with offset & link	$X[rd] \leftarrow PC + 4$; $PC \leftarrow (X[rs1] + \text{sext}(imm)) \& \sim 1$	imm ≠ value rd=X1 if rd omitd
jr rs1	unconditional branch to register address	$PC \leftarrow X[rs1]$	
la rd,symbol	load absolute address of symbol	$X[rd] \leftarrow \&\text{symbol}$	
lb rd,imm(rs1)	load byte	$X[rd] \leftarrow \text{sext}(M[X[rs1] + \text{sext}(imm)] [7:0])$	
lbu rd,imm(rs1)	load byte unsigned	$X[rd] \leftarrow \text{zext}(M[X[rs1] + \text{sext}(imm)] [7:0])$	
lh rd,imm(rs1)	load halfword	$X[rd] \leftarrow \text{sext}(M[X[rs1] + \text{sext}(imm)] [15:0])$	
lhu rd,imm(rs1)	load halfword unsigned	$X[rd] \leftarrow \text{zext}(M[X[rs1] + \text{sext}(imm)] [15:0])$	
li rd,imm	load immediate	$X[rd] \leftarrow imm$	
lw rd,imm(rs1)	load word into register	$X[rd] \leftarrow M[X[rs1] + \text{sext}(imm)] [31:0]$	
lui rd,imm	load upper immediate	$X[rd] \leftarrow imm < 12$ (shifting not done in ALU)	
mret	machine mode exception return	$PC \leftarrow \text{CSR}[mepc]$; $\text{CSR}[mstatus(mie)] \leftarrow mstatus(mpie)$	
mv rd,rs1	move	$X[rd] \leftarrow X[rs1]$	
neg rd,rs2	negate	$X[rd] \leftarrow \sim X[rs2]$	
nop	no operation	nada ($PC \leftarrow PC + 4$)	
not rd,rs2	ones complement	$X[rd] \leftarrow \sim X[rs2]$	
or rd,rs1,rs2	bitwise inclusive OR	$X[rd] \leftarrow X[rs1] X[rs2]$	
ori rd,rs1,imm	bitwise inclusive OR immediate	$X[rd] \leftarrow X[rs1] \text{sext}(imm)$	
ret	return from subroutine	$PC \leftarrow X1$	
sb rs2,imm(rs1)	store byte in memory	$M[X[rs1] + \text{sext}(imm)] \leftarrow X[rs2][7:0]$	
seqz rd,rs1	set if equal to zero	$X[rd] \leftarrow (X[rs1] == 0) ? 1 : 0$	
sgtz rd,rs2	set if greater than zero	$X[rd] \leftarrow (X[rs2] >_s 0) ? 1 : 0$	
sh rs2,imm(rs1)	store halfword in memory	$M[X[rs1] + \text{sext}(imm)] \leftarrow X[rs2][15:0]$	
sw rs2,imm(rs1)	store word	$M[X[rs1] + \text{sext}(imm)] \leftarrow X[rs2]$	
sll rd,rs1,rs2	logical shift left	$X[rd] \leftarrow X[rs1] << X[rs2][4:0]$	
slli rd,rs1,imm	logical shift left immediate	$X[rd] \leftarrow X[rs1] << imm[4:0]$	
slt rd,rs1,rs2	set if less than	$X[rd] \leftarrow (X[rs1] <_s X[rs2]) ? 1 : 0$	
slti rd,rs1,imm	set if less than immediate	$X[rd] \leftarrow (X[rs1] <_s \text{sext}(imm)) ? 1 : 0$	
sltiu rd,rs1,imm	set if less than immediate unsigned	$X[rd] \leftarrow (X[rs1] <_u \text{sext}(imm)) ? 1 : 0$	
sltu rd,rs1,rs2	set if less than unsigned	$X[rd] \leftarrow (X[rs1] <_u X[rs2]) ? 1 : 0$	
sltz rd,rs1	set if less than zero	$X[rd] \leftarrow (X[rs1] <_s 0) ? 1 : 0$	
snez rd,rs2	set if not equal to zero	$X[rd] \leftarrow (X[rs2] \neq 0) ? 1 : 0$	
sra rd,rs1,rs2	arithmetic shift right	$X[rd] \leftarrow X[rs1] >>_s X[rs2][4:0]$	
srai rd,rs1,imm	arithmetic shift right immediate	$X[rd] \leftarrow X[rs1] >>_s imm[4:0]$	
srl rd,rs1,rs2	logical shift right	$X[rd] \leftarrow X[rs1] >> X[rs2][4:0]$	
srli rd,rs1,imm	logical shift right immediate	$X[rd] \leftarrow X[rs1] >> imm[4:0]$	
sub rd,rs1,rs2	subtract	$X[rd] \leftarrow X[rs1] - X[rs2]$	
xor rd,rs1,rs2	exclusive OR	$X[rd] \leftarrow X[rs1] \wedge X[rs2]$	
xori rd,rs1,imm	exclusive OR immediate	$X[rd] \leftarrow X[rs1] \wedge \text{sext}(imm)$	

(shading indicates pseudoinstructions) [[sext(imm) values are formed from the table below based on instruction type of given instruction]]

Immediate Format Transformations How to transform immediate values encoded in the instruction's field codes into 32-bit values. These values are relative and are signed numbers.	311110540 ir[31]ir[30:25]ir[24:20] I_immed
	311110540 ir[31]ir[30:25]ir[11:7] S_immed
	311211105410 ir[31]ir[7]ir[30:25]ir[11:8]0 B_immed
	3112110 ir[31:12]0 U_immed
	31201912111010 ir[31]ir[19:12]ir[20]ir[30:21]0 J_immed

How to transform immediate values encoded in the instruction's field codes into 32-bit values. These values are relative and are signed numbers.

Instruction Formats

31	25	24	20	19	15	14	12	11	7	6	0	
func7		rs2		rs1		func3		rd		opcode		R-type
imm[11:0]				rs1		func3		rd		opcode		I-type
imm[11:5]		rs2		rs1		func3		imm[4:0]		opcode		S-type
imm[12,10:5]		rs2		rs1		func3		imm[4:1,11]		opcode		B-type
		imm[31:12]						rd		opcode		U-type
		imm[20,10:1,11,19:12]						rd		opcode		J-type

RISC-V Base Instruction Set

imm[31:12]				rd	0110111	LUI	U
imm[31:12]				rd	0010111	AUIPC	U
imm[20,10:1,11,19:12]				rd	1101111	JAL	J
imm[11:0]		rs1	000	rd	1100111	JALR	I
imm[11:0]		rs1	000	rd	0000011	LB	I
imm[11:0]		rs1	001	rd	0000011	LH	I
imm[11:0]		rs1	010	rd	0000011	LW	I
imm[11:0]		rs1	100	rd	0000011	LBU	I
imm[11:0]		rs1	101	rd	0000011	LHU	I
imm[11:0]		rs1	000	rd	0010011	ADDI	I
imm[11:0]		rs1	010	rd	0010011	SLTI	I
imm[11:0]		rs1	011	rd	0010011	SLTIU	I
imm[11:0]		rs1	110	rd	0010011	ORI	I
imm[11:0]		rs1	100	rd	0010011	XORI	I
imm[11:0]		rs1	111	rd	0010011	ANDI	I
0000000	*imm[4:0]	rs1	001	rd	0010011	SLLI	I
0000000	*imm[4:0]	rs1	101	rd	0010011	SRLI	I
0100000	*imm[4:0]	rs1	101	rd	0010011	SRAI	I
imm[12,10:5]	rs2	rs1	000	imm[4:1,11]	1100011	BEQ	B
imm[12,10:5]	rs2	rs1	001	imm[4:1,11]	1100011	BNE	B
imm[12,10:5]	rs2	rs1	100	imm[4:1,11]	1100011	BLT	B
imm[12,10:5]	rs2	rs1	101	imm[4:1,11]	1100011	BGE	B
imm[12,10:5]	rs2	rs1	110	imm[4:1,11]	1100011	BLTU	B
imm[12,10:5]	rs2	rs1	111	imm[4:1,11]	1100011	BGEU	B
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB	S
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH	S
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW	S
0000000	rs2	rs1	000	rd	0110011	ADD	R
0100000	rs2	rs1	000	rd	0110011	SUB	R
0000000	rs2	rs1	001	rd	0110011	SLL	R
0000000	rs2	rs1	010	rd	0110011	SLT	R
0000000	rs2	rs1	011	rd	0110011	SLTU	R
0000000	rs2	rs1	100	rd	0110011	XOR	R
0000000	rs2	rs1	101	rd	0110011	SRL	R
0100000	rs2	rs1	101	rd	0110011	SRA	R
0000000	rs2	rs1	110	rd	0110011	OR	R
0000000	rs2	rs1	111	rd	0110011	AND	R
csr		rs1	001	rd	1110011	CSRRW	sys
csr		rs1	011	rd	1110011	CSRRC	sys
csr		rs1	010	rd	1110011	CSRRS	sys
0011000	01000	0000	000	00000	1110011	MRET	sys

```

#-----
# Example Interrupt Driven Program.
#
# Description: The program blinks an LED. Each time the program
# receives an interrupt the code changes the state of the LED. We
# assume some external device has configured the hardware such that
# the MCU can receive an interrupt signal from an external device.
#-----
.text
main:
init:  li    x15,0x1100C004 # put output address into register
      la     x6,ISR        # load address of ISR into x6
      csrrw  x0,mstvec,x6  # store address as interrupt vector CSR[mstvec]

      mv     x8,x0         # clear x8; use as flag
      mv     x20,x0        # keep track of current output value
      sw     x20,0(x15)    # put LEDs in known state

unmask: li    x10,0x8       # set bit[3] value in x10
      csrrs  x0,mstatus,x10 # enable interrupts: set MIE in CSR[mstatus]

loop:  nop          # do nothing (easier to see in simulator)
      beq     x8,x0,loop  # wait for interrupt

      xori    x20,x20,1    # toggle current LED value
      sw     x20,0(x15)    # output LED value

      mv     x8,x0         # clear flag
      csrrs  x0,mstatus,x10 # enable interrupts: set MIE (bit3) in CSR[mstatus]
      j      loop         # return to loopville
#-----

#-----
#- The ISR: sets bit x8 to act as flag to task code.
#-----
ISR:   li     x8,0x1       # set flag to non-zero

      li     x9,0x80       # set bit7 in x9
      csrrc  x0,mstatus,x9 # clear bit7 (MPIE) in CSR[mstatus]

done:  mret              # return from interrupt
#-----

```

Preferred Register Usage

Register type	Calling Code	Subroutine (SR)		Comments
		Can SR change register values?	Should SR save register values?	
Argument (a0-a7)	SR may change regs; calling code must save regs before call if calling code relies on these reg values	Yes	Not necessary; SR can change when passing data to calling code.	These regs change based on how the calling code and SR pass and/or return data
Saved (s0-s11)	Calling code relies on SR not permanently changing these regs	Yes	Must save & restore if SR changes reg values	Save to stack or to unused Temporary registers
Temporary (t0-t6)	SR may change regs; calling code must save regs if calling code relies on these values not changing.	Yes	No	Working regs for subroutine

31	30	23	22					8	7		4	3		0	
n/a	res			n/a				MPIE	res	n/a	res	MIE	res	n/a	res

The CSR[mstatus] Register